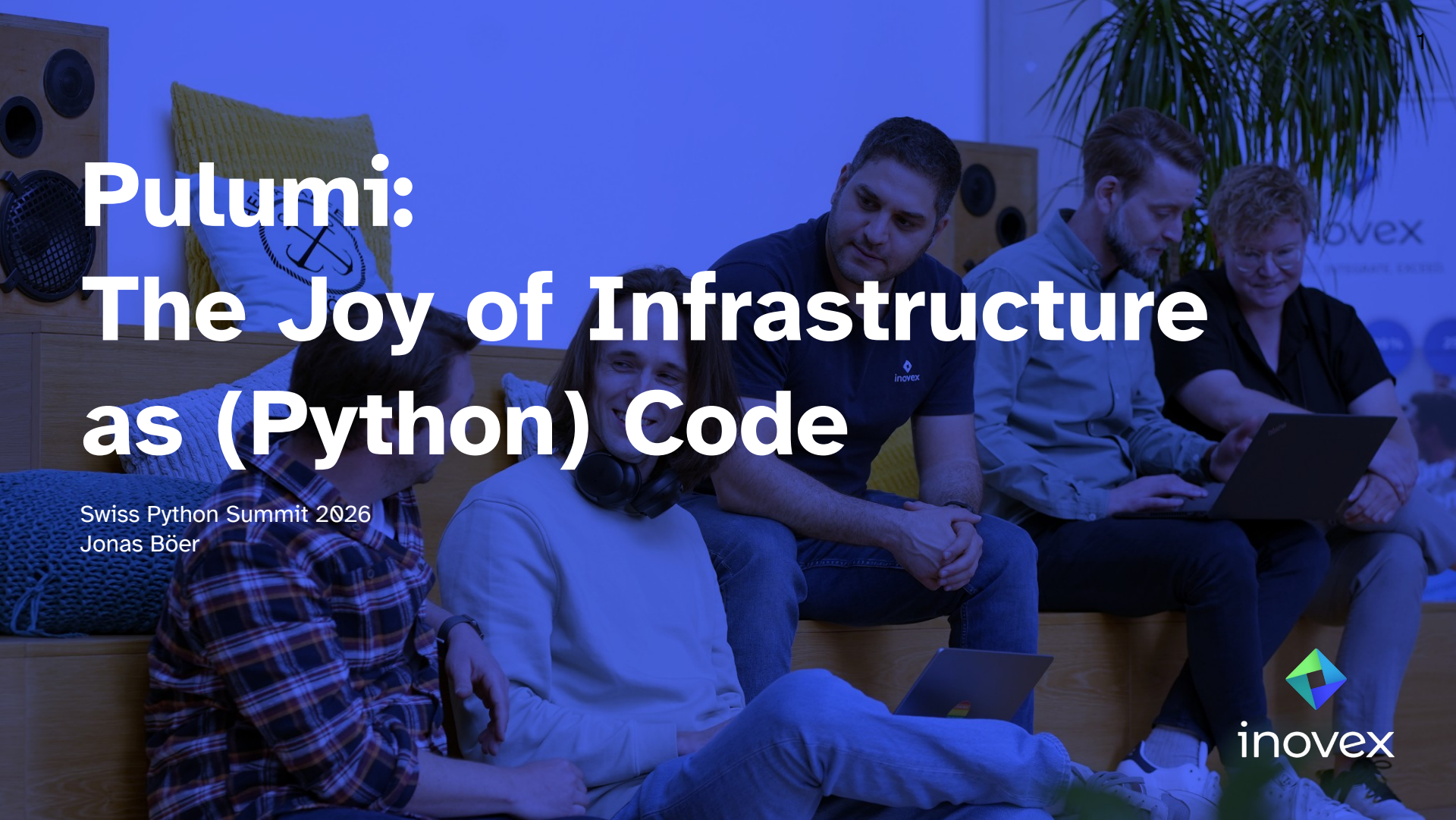


Pulumi: The Joy of Infrastructure as (Python) Code

Swiss Python Summit 2026

Jonas Böer

inovex

WE ARE INOVEX – INNOVATE. INTEGRATE. EXCEED.

Your Partner for Digital Projects and Products with a broad portfolio of services

- **Data & AI**
- **Application Development**
- **Scalable IT Infrastructures**
- **Training & Coaching**

Exclusively Permanent Employees

Growth through Innovation and Successful Projects

inovex

SOFTWARE · DATA & AI · INFRASTRUCTURE



inovex.swiss

Quick Recap:
What is Infrastructure as Code?

- No manual creation of resources, they are defined in code
- Code as Single Source of Truth
  - Reviews
  - Versioning
  - Automated Deployment
- Declarative, not Imperative

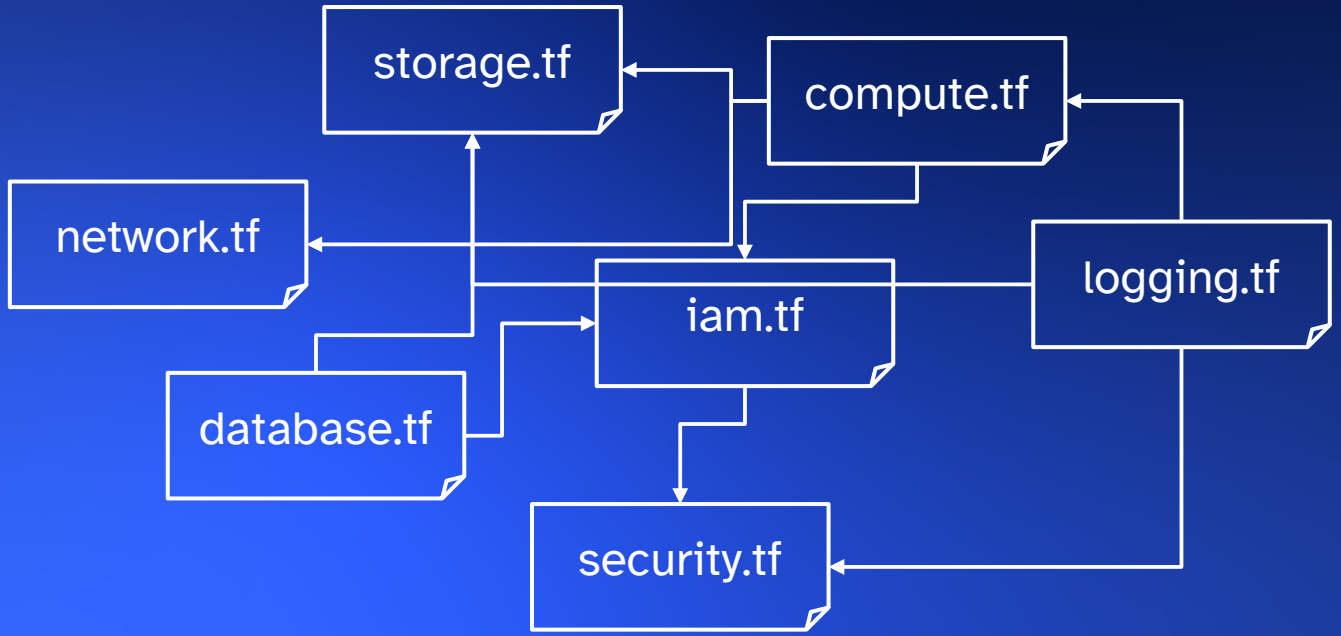
- Frequently hard to read

providers.tf

main.tf

variables.tf

outputs.tf





Pulumi

The Solution for Everything?

```python
from pulumi_kubernetes.apps.v1 import Deployment
from pulumi_kubernetes.core.v1 import Service

app_labels = {"app": "hello-world"}

deployment = Deployment(
    "sample-kubernetes-deployment",
    spec={
        "selector": {"match_labels": app_labels},
        "template": {
            "metadata": {"labels": app_labels},
            "spec": { "containers": [{
                        "name": "hello-world",
                        "image": "nginxdemos/hello:latest",
                        "ports": [{"container_port": 80}]
            }] },
        },
    }
)
service = Service(
    "sample-kubernetes-service",
    spec={
        "type": "NodePort",
        "selector": app_labels,
        "ports": [{
                "port": 80,
                "target_port": 80,
                "node_port": 30080
        }],
    }
)

pulumi.export("deployment_name", deployment.metadata.name)
pulumi.export("service_name", service.metadata.name)
```

Pulumi

python™

- One Object = One Resource
- Regular Python Code
- Describe the target state, not the way there

- Pulumi is published under Apache License
- Bindings for Python and other languages
- Resource Providers for many services

```python
service = Service(
    "sample-kubernetes-service",
    spec=ServiceSpecArgs(
        type="NodePort",
        selector=app_labels,
        ports=[
            ServicePortArgs(
                port=80,
                target_port=80,
                node_port=30080,
            )
        ],
```

```
> pulumi up
[ ... ]
Updating (sps):
     Type                                 Name                          Status
 +   pulumi:pulumi:Stack                  pulumi-k8s-example-sps        created (10s)
 +   ├─ kubernetes:core/v1:Service        sample-kubernetes-service     created (10s)
 +   └─ kubernetes:apps/v1:Deployment     sample-kubernetes-deployment  created (1s)

Outputs:
    deployment_name: "sample-kubernetes-deployment-968c0192"
    service_name   : "sample-kubernetes-service-705d8825"
Resources:
    + 3 created

Duration: 11s
```

```python
class SimpleApp(pulumi.ComponentResource):
    def __init__(self, name: str, image: str, opts: pulumi.ResourceOptions | None = None) -> None:
        super().__init__("inovex:sps-example:SimpleApp", name, {}, opts)

        child_opts = pulumi.ResourceOptions(parent=self)
        app_labels = {"app": name}

        self.deployment = Deployment(f"{name}-deployment",
            spec={
                ...,
                "template": {
                    "metadata": {"labels": app_labels},
                    "spec": {
                        "containers": [{"name": f"{name}-server", "image": image, "ports":  ...}]
                    },
                },
            },
            opts=child_opts,
        )

        self.service = Service( ... )

        self.deployment_name = self.deployment.metadata.name
        self.service_name = self.service.metadata.name
        self.register_outputs({"deployment_name": self.deployment_name, "service_name": self.service_name})

app = SimpleApp("hello-world-cr", image="nginxdemos/hello:latest")
pulumi.export("deployment_name", app.deployment_name)
pulumi.export("service_name", app.service_name)
```

```
> pulumi up
[ ... ]
Updating (sps):
     Type                                 Name                          Status
 +   pulumi:pulumi:Stack                  pulumi-k8s-example-sps        created (10s)
 +   └─ inovex:sps-example:SimpleApp      hello-world-cr                created (10s)
 +      ├─ kubernetes:core/v1:Service     hello-world-cr-service        created (10s)
 +      └─ kubernetes:apps/v1:Deployment  hello-world-cr-deployment     created (1s)

Outputs:
    deployment_name: "hello-world-cr-deployment-89fd689e"
    service_name   : "hello-world-cr-service-b0578b4f"

Resources:
    + 4 created

Duration: 11s
```

```
applications.yaml
---

apps:
  - name: "nginxdemo"
    image: "nginxdemos/hello:0.4"
  - name: "hello-k8s"
    image: "paulbouwer/hello-kubernetes:1.10"
  - name: "ealen-echo"
    image: "ealen/echo-server:0.9.2"
```

```python
from simple_app import SimpleApp

with open("applications.yaml") as config:
    app_config = yaml.safe_load(config)
    for app in app_config["apps"]:
        apps.append(
            SimpleApp(
                name=app["name"],
                image=app["image"],
            )
        )
```

- Separate Config and Code
- Reduction of Boilerplate Code
- More complex modelling possible

```python
from pulumi import automation as auto

def pulumi_program():
    app = SimpleApp("hello-world", image="ealen/echo-server:latest")

    pulumi.export("deployment_name", app.deployment_name)
    pulumi.export("service_name", app.service_name)


def main():
    action = "up"
    stack_name = "dev"
    project_name = "Pulumi-Vortrag"

    stack = auto.create_or_select_stack(
        stack_name=stack_name,
        project_name=project_name,
        program=pulumi_program,
    )

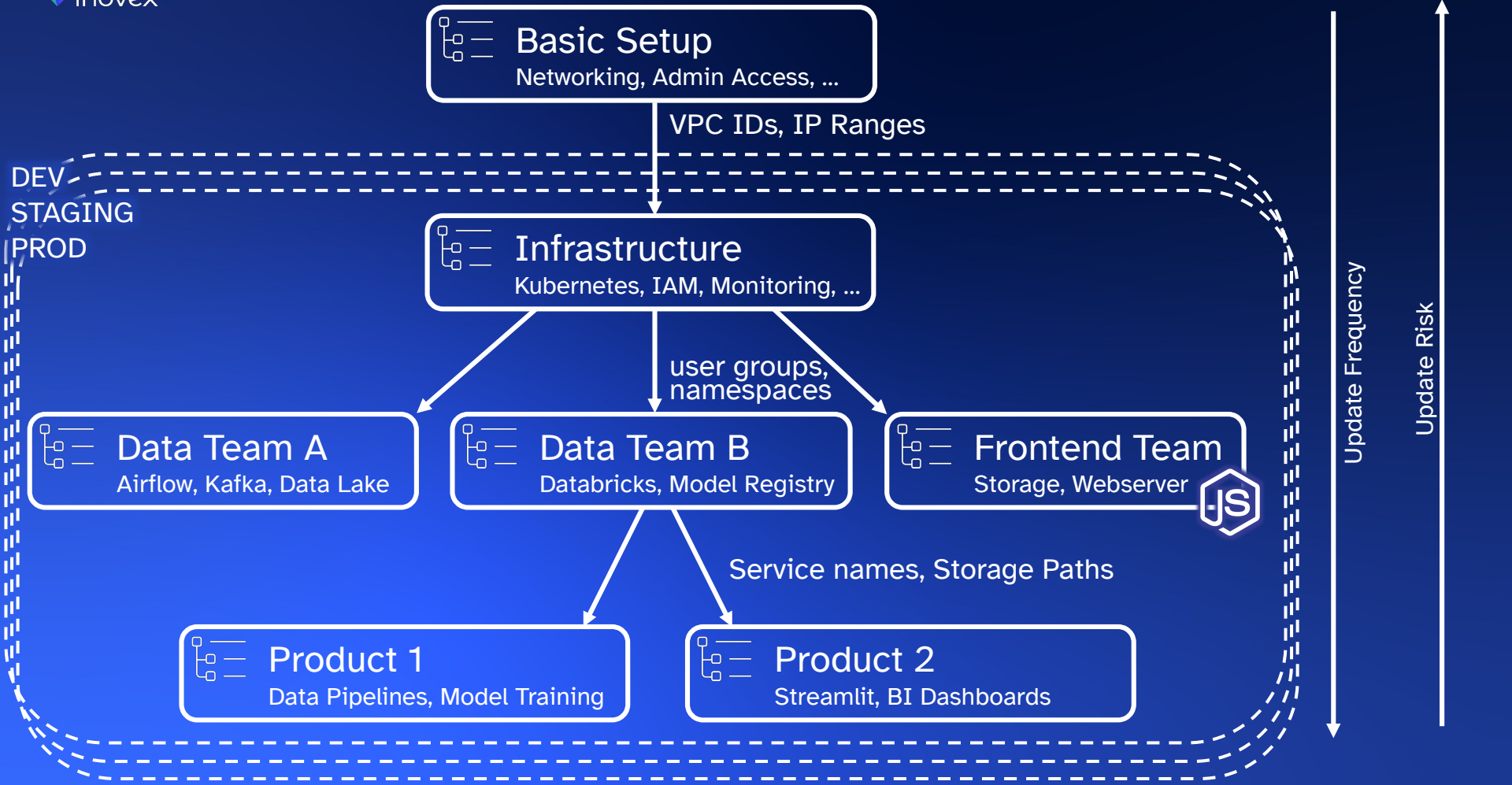
    if action == "preview":
        res = stack.preview(on_output=print)
    elif action == "up":
        up_res = stack.up(on_output=print)


if __name__ == "__main__":
    main()
```

Pulumi

Automation API

- The program calls Pulumi, not vice versa
- Manual Stack selection
- Everything can be configured on the fly
- Especially for infrastructure teams
- Enables Multi-Stack architecture


Basic Setup
Networking, Admin Access, ...
VPC IDs, IP Ranges
DEV
STAGING
PROD
Infrastructure
Kubernetes, IAM, Monitoring, ...
user groups,
namespaces
Data Team A
Airflow, Kafka, Data Lake
Data Team B
Databricks, Model Registry
Frontend Team
Storage, Webserver
Service names, Storage Paths
Product 1
Data Pipelines, Model Training
Product 2
Streamlit, BI Dashboards
Update Frequency
Update Risk

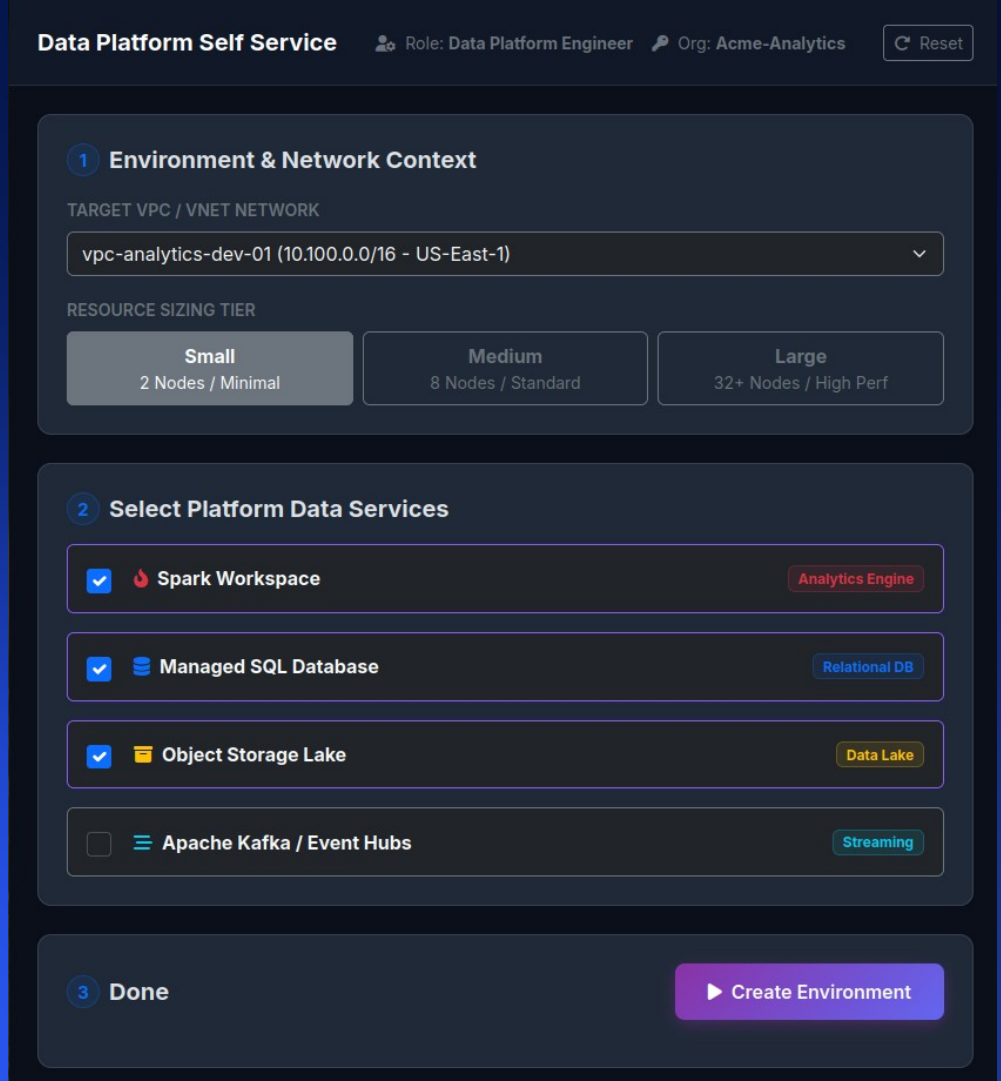
A few more things...

Tests for your Infrastructure Code

Write your own Providers

k8s Custom Resource Definitions

Stack Versioning



When would you NOT use Pulumi?

- Your IaC structure is already complex and mature
- You don’t need the Automation API
- Your teams don’t deploy any infrastructure themselves
- Software Engineering for your Infrastructure is a bit of work
- The stack management is too complex for your organization

So how is  Pulumi ?

- Fun
- Read and write your infrastructure like your code
  - THE IaC solution for DevOps
- Software Engineering principles for Infrastructure required
- So many possibilities to design your infrastructure
- So many possibilities to shoot yourself in the foot